Campus Santa Fe

Database modelling exercise for the challenge

Software Construction and Decision Making

Estudiante:

Juan Pablo Briñez Corzo

Santiago Martin Vences

José Paulo Vélez Ortiz

Matrícula:

A01782050

A01787450

A01787207

Fecha de entrega:

28 de abril

Diagram justification

The model made for the game “Return” is composed of 9 main tables that represent the main aspects needed for the game to work. On the other hand, we also have 3 intermediate tables that help us avoid redundancy in our game. In this way, we can keep our database organized and functional.

Database tables justification

Users: This table contains information about a user who creates an account in our game. It is important because it will help us register all the different users that access our game, which will be too many to store them on the code. Each row has a unique identifier and the next data:

- username: Indicates the nickname that the user used to create the account
- email: Indicates the address used to create an account
- password hash: Indicates the password of the user. It is hashed to prevent security issues related to the DB
- created at: Indicates the date when the account was created

This table has the following relationships:

- Player profiles: One user can have many player profiles, but many users can’t have the same profile. (1:M)

Player profiles: This table contains information about a character that a user creates in the game. This table is important because it will register the characters a user makes, so players can use them at any time without having to save them on their personal computer. Each row has a unique identifier and the next data:

- user id: Identifier that references the user who created that player profile

- archetype: Indicates the character type the player profile has (ej, mage, warrior, etc.)
- total experience: Indicates the total experience that the player profile currently has
- level: Indicates the level the player profile has achieved until now
- attribute points: Indicates the quantity of attribute points a player has accumulated to upgrade their stats
- created at: Indicates the date when the player profile was created

This table contains the following relationships:

- Users: One user can have many player profiles, but many users can't have the same profile. (1:M)
- Player global stats: One player profile can have multiple stats, but multiple player profiles won't have the same stats. (1:M)
- Attributes: A player profile can have many attribute distributions over the time that the player profile has existed; multiple player profiles won't have the same attribute distributions over time. (1:M)
- Cards: A player profile can have many cards in its inventory, and many player profiles can have the same card in their inventory. (M: N)
- Runs: A player profile can have many runs, but many player profiles won't repeat the same run (especially because it is a roguelike). (1:M)

Player global stats: This table contains information about the stats a player's profile has accumulated since it was created. This table is important because it will keep a record of the stats a player achieves with a profile, which is useful for players to know things like their parry success rate or if they have completed the collection of cards in the game. Each row contains a unique identifier and the next information:

- player id: References the player profile that has the stats contained in the row
- total runs: Indicates the total number of runs a player has had over the creation of his account
- total victories: Indicates the total number of times a player has completed a game with a player profile

- best completion time seconds: Indicates the best time you have achieved in the runs where you completed the game
- total perfect parries: Indicates the quantity of perfect parries a player profile has achieved since the creation of his account
- total normal parries: Indicates the quantity of normal parries a player profile has achieved since the creation of his account
- total poor parries: Indicates the quantity of poor parries a player profile has achieved since the creation of his account
- total enemies defeated: Indicates the total number of enemies defeated by a player profile since it was created
- total bosses defeated: Indicates the total number of bosses defeated by a player profile since it was created
- total cards collected: Indicates the total number of cards collected since the creation of the player profile
- updated at: Indicates the last time this row was updated

This table contains the following relationships:

- Player profiles: One player profile can have multiple stats, but multiple player profiles won't have the same stats. (1:M)

Attributes: This table contains information about the attributes a player profile currently has. This table is important because it keeps track of the attributes and their changes within a player profile over time. Each row has a unique identifier and the next information:

- player id: Contains information about the player profile that has the attributes in the table
- Strength: Indicates the amount of extra physical attack a player profile will have
- vigor: Indicates the amount of extra life points a player profile will have
- intelligence: Indicates the amount of extra magic attack a player profile will have
- endurance: Indicates the amount of extra stamina a player profile will have

- dexterity: Indicates the amount of speed a player profile will have
- updated at: Indicates when these attributes were updated

This table contains the following relationships:

- Player profiles: A player profile can have many attribute distributions over the time that the player profile has existed; multiple player profiles won't have the same attribute distributions over time. (1:M)

Runs: This table contains information about the runs a player profile will have when playing the game. This table is important because it keeps track of things like the time each run took or the final floor reached, which can then be presented to a player. Each row has a unique identifier and the next information:

- start time: Indicates the date when a run started
- end time: Indicates the date when a run ended
- Completion time, seconds: Indicates the time it took to complete a run in seconds
- Final floor reached: Indicates the floor where the player finished their run
- Final room reached: Indicate the room where the player finished their run
- victory: Indicates if the player finished the game or not
- Death cause: Indicates how the player died in the run
- Permanent card ID: Indicates which card was chosen after finishing a run

This table contains the following relationships:

- Player profiles: A player profile can have many runs, but many player profiles won't repeat the same run (especially because it is a roguelike). (1:M)
- Room sessions: A run can have multiple room sessions, but the same room session can't be repeated across multiple runs. (1:M)
- Cards: One or zero cards can be selected as permanent in multiple runs, but multiple cards can't be chosen on a run. (0..1:M)

- Player cards: A single run can be referenced by many player card entries (one per card obtained during it), but each player card entry belongs to only one run. (1:M)

Player cards: This table contains information about which cards are contained in which player profiles. It is useful because it maps the many-to-many relationships between the cards and player profiles, which can be visualized as the inventory multiple player profiles have in the game. Each row has a unique id and the next information:

- player id: Indicates the player profile that has that card
- card id: Indicates the card that is related to the player profile
- obtained at run: Indicates the run where a card was obtained in a player profile
- is permanent: Indicates if the player will stay after the run finishes or if it will vanish

As it was mentioned, this table maps the many-to-many relationship between player profiles and cards, which is why player profiles and cards have a 1:M relationship with this table.

The table also contains the following relationships:

- Runs: A player card entry belongs to only one run (the one where it was obtained), but a single run can be referenced by many player card entries. (M:1)

Cards: This table contains information about the cards that will be available throughout the game. This table will help us define all the cards that will be available, and will be important to define the inventory the player will have, and the deck the player is going to use in every fight. The table contains a unique identifier and the next information:

- name: Indicates the name of the card
- description: Indicates information about what the cards do, it can be figurative
- action type: Defines if the action will be to attack or to defend
- stamina cost: Indicates how much stamina the card will consume after one use
- base damage: Indicates how much damage the card will do on its own
- rarity: Indicates how rare the card is (How difficult it is to obtain)
- scales with: Indicates what attribute the player needs to increase in their profile to make the card better
- scaling factor: Indicates in which proportion the card will scale after increasing a point of the attribute present in the scales
- required attribute: Indicates the necessary attributes to use the card
- is boss reward: Indicates if the card is a boss reward or not

This table contains the following relationships:

- Runs: Zero or one card can be chosen as the permanent reward for multiple runs, but each run selects at most one permanent card. (0..1:M)
- Player: A player profile can have many cards in its inventory, and many player profiles can have the same card in their inventory. (M: N)
- Room sessions (Card reward): Zero or one card can be the reward for multiple room sessions. However, a one-room session can't have multiple cards as a reward. (0..1:M)
- Room sessions (Deck): One card can be in multiple room sessions in the deck, while multiple cards can be in one room session for the deck. (M: N)

Room sessions: Contains information about the combat sessions per room. This table is important because it maps each combat that has occurred in a room, as there will be multiple fights in a room. Each row has a unique ID and the next information:

- run id: Indicates the run this room session occurred in
- room id: Indicates the room where the session occurred
- start time: Indicates the date when the room session started

- end time: Indicates the date when the room session finished
- result: Indicates if the player won or lost in the room session
- waves survived: Indicates how many waves have been survived up until that room session
- Experience gained: Indicates how much experience was gained from that room session
- Card reward ID: Indicates the ID of the reward card received in that session

This table has the following relationships:

- Cards (reward card): Zero or one card can be the reward for multiple room sessions. However, a one-room session can't have multiple cards as a reward. (0..1:M)
- Cards (Deck): One card can be in multiple room sessions in the deck, while multiple cards can be in one room session for the deck. (M: N)
- Runs: A run can have multiple room sessions, but the same room session can't be repeated across multiple runs. (1:M)
- Parry stats: One room session has one set of parry stats, and those stats belong to one room session only. (1:1)
- Enemies: Multiple enemies can be in a room session, while a single enemy can also appear in multiple room sessions. (M: N)
- Room: One room can have multiple room sessions, but the same room session can't repeat in different rooms. (1:M)

Active deck cards: This table contains information about the cards that are in a deck for a room session. It is important because this table allows us to map the many-to-many relationship between room sessions and cards to make a deck. Each row contains a unique identifier and the next information:

- room session ID: Indicates the room session where a card was present
- card ID: Indicates the card that was present in a room session
- slot: Indicates the slot of a card present in a room session

As it was mentioned, this table maps the many-to-many relationship between room sessions and cards, which is why room sessions and cards have a 1:M relationship with this table.

This table also contains no other relationships

Parry stats: This table contains information about the parry stats that a player has after every room session. It is important to have this table because in this way the game will be able to show these stats to the player, which will tell him what he has to improve. Each row contains a unique identifier and the next information:

- Session ID: Indicates the session corresponding to these parry stats
- Perfect parries: Indicates the number of perfect parries done in a room session
- Normal parries: Indicates the number of normal parries done in a room session
- Poor parries: Indicates the number of poor parries done in a room session
- Parries missed: Indicates the number of parries that a player missed during a room session

This table has the following relationships:

- Room sessions: Each set of parry stats belongs to exactly one room session, and one room session has exactly one parry stats record. (1:1)

Enemies: This table contains information about all the enemies that will be in the game. This table is important because if it didn’t exist, we would have to define the enemies directly in the game programming, which would be inefficient. Each row contains a unique identifier and the next information:

- Name: Indicates the name of the enemy
- Health min: Indicates the minimum amount of health an enemy can have
- Max health: Indicates the maximum amount of health an enemy can have

- Physical damage min: Indicates the minimum amount of physical damage an enemy can have
- Physical damage max: Indicates the maximum amount of physical damage an enemy can have
- Magic damage min: Indicates the minimum amount of magic damage an enemy can have
- Magic damage max: Indicates the maximum amount of magic damage an enemy can have
- Physical defense: Indicates the damage reduction that a player's physical attack will have
- Magic defense: Indicates the damage reduction that a player's magic attack will have
- Xp reward: Indicates the amount of XP that an enemy will give to a player after being defeated
- Is boss: Indicates if an enemy is a boss or not
- Floor ID: Indicates the floor this enemy can appear on

This table has the following relationships:

- Room sessions: Multiple enemies can be in a room session, while a single enemy can also appear in multiple room sessions. (M: N)
- Floors: Zero or one floor can have multiple enemies, but one enemy can't be on multiple floors. (0..1:M)
- Enemy abilities: One enemy can have multiple abilities, but many enemies can't have the same ability. (1:M)

Enemy abilities: This table maps the abilities that enemies will have in the game. This table is important because enemies will have a lot of different abilities in the game, which is why they have to be stored in the database. Each row has a unique identifier and the next information:

- enemy ID: Indicates the enemy that has this movement
- name: Indicates the name of the ability
- description: Gives a short description of the ability

- damage type: Indicates if the damage is physical or magic

This table has the following relationships:

- Enemies: One enemy can have multiple abilities, but many enemies can't have the same ability. (1:M)

Floors: This table contains information about the floors that will be in the game. This table is important because if in the future we want to have more floors, with this table we make sure we will have the space to do it efficiently. Each row in this table has a unique identifier and the next information:

- floor number: Indicates the floor number of the floor
- name: Indicates the name of the floor
- ambience: Indicates the ambience of the floor
- color palette: Indicates the colors that will be used in the floor

This table has the following relationships:

- Enemies: Zero or one floor can have multiple enemies, but one enemy can't be on multiple floors. (0..1:M)
- Rooms: One floor can have many rooms, but a room can't repeat across floors. (1:M)

Rooms: This table contains information about the rooms in the game. This table is important because it describes the rooms that will be available on each floor. Each row has a unique identifier and the next information:

- floor id: Indicates the floor where the room is
- room number: Indicates the number of the room in each floor
- is boss: Indicates if the room has a boss inside it

This table has the following relationships:

- Room sessions: One room can have multiple room sessions, but the same room session can't repeat in different rooms. (1:M)

Session enemies defeated: This table contains information about enemies defeated in each room session. This table is important because it maps the many-to-many relationship between enemies and the room session. Each row has a unique identifier and the next information:

- session ID: Indicates the room session where the enemy was defeated
- enemy ID: Indicates the enemy that was defeated in the session
- wave number: Indicates the wave number of the enemy that appeared in the session
- defeated at: Indicates the room the player was defeated in

As it was mentioned, this table maps the many-to-many relationship between room sessions and enemies, which is why room sessions and enemies have a 1:M relationship with this table.

This table also contains no other relationships

Integrity constraints

The primary keys that guarantee that every register in our database is unique are:

- users: (id INT PRIMARY KEY NOT NULL)
- player_profiles: (id INT PRIMARY KEY NOT NULL)
- attributes: (id INT PRIMARY KEY NOT NULL)
- player_global_stats: (id INT PRIMARY KEY NOT NULL)
- runs: (id INT PRIMARY KEY NOT NULL)
- parry_stats: (id INT PRIMARY KEY NOT NULL)
- cards: (id INT PRIMARY KEY NOT NULL)
- room_sessions: (id INT PRIMARY KEY NOT NULL)
- rooms: (id INT PRIMARY KEY NOT NULL)
- floors: (id INT PRIMARY KEY NOT NULL)
- enemies: (id INT PRIMARY KEY NOT NULL)
- enemy_abilities: (id INT PRIMARY KEY NOT NULL)
- session_enemies_defeated: (id INT PRIMARY KEY NOT NULL)
- active_deck_cards: (id INT PRIMARY KEY NOT NULL)
- player_cards: (id INT PRIMARY KEY NOT NULL)

These keys will be set to auto increment to ensure there is a logic sequence in the database registers.

Also, each one of these keys have a restriction set to be not null, to guarantee entity integrity throughout the tables of the database. The same thing happens for other values in the database, in our case the foreign keys because we want to avoid unnecessary data and guarantee integrity. The foreign keys that are on the schema are:

- player_profiles
    - user_id INT, FOREIGN KEY(user_id) REFERENCES users(id)

- attributes
  - player_id INT, FOREIGN KEY(player_id) REFERENCES player_profiles(id)

- player_cards
  - player_id INT, FOREIGN KEY(player_id) REFERENCES player_profiles(id)
  - card_id INT, FOREIGN KEY(card_id) REFERENCES cards(id)
  - obtained_at_run INT, FOREIGN KEY(obtained_at_run) REFERENCES runs(id)

- rooms
  - floor_id INT, FOREIGN KEY(floor_id) REFERENCES floors(id)

- enemies
  - floor_id INT, FOREIGN KEY(floor_id) REFERENCES floors(id)

- enemy_abilities
  - enemy_id INT, FOREIGN KEY(enemy_id) REFERENCES enemies(id)

- runs
  - player_id INT, FOREIGN KEY(player_id) REFERENCES player_profiles(id)
  - permanent_card_chosen_id INT, FOREIGN KEY(permanent_card_chosen_id) REFERENCES cards(id)

- room_sessions
  - run_id INT, FOREIGN KEY(run_id) REFERENCES runs(id)
  - room_id INT, FOREIGN KEY(room_id) REFERENCES rooms(id)
  - card_reward_id INT, FOREIGN KEY(card_reward_id) REFERENCES cards(id)

- active_deck_cards
  - room_session_id INT, FOREIGN KEY(room_session_id) REFERENCES room_sessions(id)
  - card_id INT, FOREIGN KEY(card_id) REFERENCES cards(id)

- session_enemies_defeated
  - session_id INT, FOREIGN KEY(session_id) REFERENCES room_sessions(id)
  - enemy_id INT, FOREIGN KEY(enemy_id) REFERENCES enemies(id)

- parry_stats
  - session_id INT, FOREIGN KEY(session_id) REFERENCES room_sessions(id)

- player_global_stats
  - player_id INT, FOREIGN KEY(player_id) REFERENCES player_profiles(id)

Finally, each timestamp present in the database will need to use the CURRENT_TIMESTAMP restriction, as it automates the process of setting the date manually through other processes.